多智能体创意涌现系统(IdeaSpark) 产品需求文档 V2.0

版本: V2.0
撰写日期: 2026年3月12日
负责人: huang suxiang
产品定位: 一人实验室科研 idea 自动生成与验证平台

---

1. 产品概述

1.1 产品背景

科研工作中，idea 的产生往往是最具挑战性的环节。传统的头脑风暴依赖多人协作，但一人实验室研究者缺乏多样化认知碰撞的环境。大语言模型(LLM)虽然强大，但单一模型的输出容易陷入单一视角和知识截止日期限制[1]。

本产品旨在通过多智能体协作架构，模拟跨学科专家团队的思维碰撞，结合实时外部知识 API，为独立研究者提供一个 24/7 可用的"虚拟智囊团"，突破单一 LLM 的局限性。

1.2 核心价值主张

- **认知多样性涌现**: 通过 4+ 差异化 Persona Agent 产生跨领域视角碰撞
- **知识实时性**: 集成学术搜索(arXiv, Semantic Scholar)和通用搜索(Tavily) API，消除知识滞后
- **零框架依赖**: 从零实现底层架构，完全可控可定制，避免框架黑盒
- **经验证的协作模式**: 采用 Graph 拓扑 + Separate-Then-Together 模式，性能优于传统 Debate/Chain 架构[2][3]

1.3 目标用户画像

用户类型	典型场景
独立研究者/博士生	需要快速生成可发表的研究方向 idea
一人实验室/创业团队	缺乏跨学科背景的协作者进行头脑风暴
科研项目负责人	需要系统化评估多个研究方向的可行性

1.4 核心技术指标

- **idea 产出量**: 单次运行产出 10-16 个初始 idea，经 2-3 轮迭代后收敛至 3-5 个高质量方案
- **领域覆盖度**: 至少覆盖 3 个不同学科视角(通过 Persona 差异化保证)
- **知识新鲜度**: 引用论文/资料的发布时间 ≥70% 在近 2 年内
- **响应时间**: 完整的 Separate-Together 两阶段 ≤5 分钟(基于 GPT-4o API)
- **API 成本**: 单次完整运行 ≤$2.00(按 2026 年 OpenAI 定价)

---

2. 系统架构设计

2.1 整体架构

系统采用无框架的模块化设计，分为四层架构[4]:

层级	职责
用户交互层	接收研究主题输入，展示 idea 演化过程和最终结果
编排调度层	实现 Separate-Together 流程控制，管理 Agent 生命周期
Agent 执行层	多个 Persona Agent 独立推理与协作通信(Graph 拓扑)
工具与知识层	LLM API 调用、外部搜索 API、黑板通信机制

2.2 核心模块定义

2.2.1 Agent 基础抽象

每个 Agent 的最小实现包含以下属性:

class Agent:
name: str # Agent 唯一标识
persona: str # 角色人格描述(系统 Prompt)
model_client: LLMClient # 统一 LLM 调用接口
memory: list[Message] # 对话历史记忆
tools: list[Tool] # 可调用的外部工具
temperature: float # 生成参数(发散型高温，批判型低温)

2.2.2 黑板通信机制(Blackboard)

所有 Agent 共享的全局信息池，支持消息的写入、读取和引用追踪:

class Message:
id: str # 消息唯一 ID
sender: str # 发送 Agent 名称
content: str # 消息正文
msg_type: str # idea | critique | synthesis | vote
references: list[str] # 引用的前序消息 ID
timestamp: datetime # 时间戳

2.2.3 工具注册表(Tool Registry)

统一管理外部 API 调用，每个工具实现标准接口:

class Tool:
name: str
description: str # 供 LLM 理解的工具说明

```
def execute(query: str) -> dict:
    # 执行具体调用逻辑
    pass
```

---

3. Persona Agent 设计

3.1 角色配置策略

基于研究表明，角色多样性直接决定 idea 的领域分布[3]。系统默认配置 5 个差异化角色:

角色	认知风格	工具绑定	温度参数
发散者 (Divergent)	类比思维，跨领域迁移	web_search, paper_search	1.0
领域专家 (Expert)	深度专业知识	paper_search, citation_graph, pdf_reader	0.5
批评者 (Critic)	挑战假设，寻找漏洞	citation_graph	0.3
连接者 (Connector)	发现关联，组合融合	无	0.7
评估者 (Evaluator)	**锦标赛对比投票**，优先级排序	无	0.3

3.2 Persona Prompt 模板

每个角色的 system prompt 包含三部分:

1. **角色定义**: 明确身份、专长、认知偏好
2. **Cognitive Planning 前置步骤**[2]: 强制在输出前进行显式推理
3. **输出格式约束**: 结构化的消息类型标记

示例 - 发散者 Prompt:

你是一名"发散者"(Divergent Thinker)，擅长跨领域类比和大胆联想。
你的核心价值是打破常规，从看似无关的领域找到启发。

在回答之前，先用 3 句话写出:

1. 我观察到其他人提出了什么关键 idea?
2. 当前最缺少的视角是什么?
3. 我接下来要做的具体贡献是什么?

你可以使用工具: web_search(最新趋势), paper_search(学术灵感)

输出格式:
[msg_type: idea]
<你的 idea 内容>

3.3 动态 Persona 扩展

用户可根据具体研究领域自定义新角色，系统提供 Persona Builder 工具生成符合规范的 Prompt。

---

4. 协作流程设计

4.1 核心流程: Separate → Together 两阶段

基于最新研究验证的最优协作模式[3]:

阶段一: 独立发散(Separate Phase)

- **触发条件**: 用户输入研究主题(如"多智能体在药物发现中的应用")
- **执行逻辑**:
  1. 发散者、领域专家并行启动，各自独立生成 5-8 个 idea
  2. 彼此看不到对方输出，避免锚定效应
  3. 调用外部工具获取最新论文和趋势(arXiv 近 2 年论文 Top 10)
  4. 输出写入黑板，标记 `msg_type = "idea"`
- **预期产出**: 10-16 个初始 idea，覆盖不同学科视角

阶段二: 协作碰撞(Together Phase - Graph Topology with Routing)

⚠ V2.0 新增: 分组讨论机制

- **通信拓扑**: Graph 结构 - 每个 Agent 都能看到所有其他 Agent 的输出[2]
- **执行逻辑**:

1. **分拣机制**: 将 10-16 个初始 idea 按语义相似度聚类为 3-4 组(每组 3-5 个 idea)
2. **分组讨论**:
   - 连接者、批评者分别负责不同组的讨论(并行执行)
   - 每组内进行 2-3 轮 Extension/Combination/Refinement
3. **全局整合**: 各组讨论结果合并，连接者进行跨组融合(msg_type = "synthesis")
4. 发散者和专家根据批评修正原始 idea(msg_type = "idea"，引用原 ID)
5. 循环 3-5 轮，直到批评减少

协作行为分类[5]:

- **Extension**(扩展): 在已有 idea 基础上加新元素
- **Combination**(组合): 融合多个 idea 为新方案
- **Refinement**(精炼): 优化细节和可行性

4.2 收敛与评估

⚠ V2.0 核心改进: 锦标赛淘汰制

传统打分机制问题:

- LLM 对绝对分数(70分、80分)的校准能力弱
- 容易受生成顺序影响(位置偏差)
- 评分标准难以统一

新方案 - Tournament Selection:

1. **两两对比**: 从所有 idea 中随机抽取 2 个，由评估者投票"A 还是 B 更好，为什么?"
2. **多轮比赛**: 每个 idea 参与 5-8 场对比
3. **Elo 积分**: 基于胜率计算 Elo 分数(初始 1500，胜利 +16，失败 -16)
4. **Top-K 筛选**: 选出 Elo 分数最高的 3-5 个 idea

评估维度(保持不变):

- 新颖性(30%): 是否突破现有研究范式
- 可行性(40%): 技术路径是否清晰

- 影响力(30%): 潜在学术/应用价值

多轮迭代:

- Top-K 进入下一轮 Separate-Together 深化
- **终止条件**: 达到预设轮数(默认 3 轮)或 idea Elo 分数收敛(变化 <5%)

4.3 流程状态机

```
[用户输入]
↓
[Separate: 并行发散]
↓
[语义聚类: 10-16 ideas → 3-4 组]
↓
[Together: 分组讨论 × 2-3 轮]
↓
[全局整合: 跨组融合]
↓
[锦标赛评估: Tournament Selection]
↓
[Top-K 筛选]
↓
判断: 达到终止条件?
├── 是 → [输出最终方案]
└── 否 → 回到 [Separate: 对 Top-K 深化]
```

---

5. 外部 API 集成方案

5.1 API 选型与职责

API	用途	调用场景	成本估算
Tavily Search	实时网络搜索	了解最新行业趋势	免费 1000 次/月
arXiv API	预印本论文搜索	获取最新学术成果	完全免费
Semantic Scholar	论文+引用图谱	验证研究脉络	免费(需注册)
PDF Reader Tool	**全文解析**	**深度阅读 Top-K 论文**	**本地运行 (Unstructured)**
OpenAI GPT-4o	主推理引擎	所有 Agent 调用	$5/1M tokens(输入)

⚠ V2.0 新增: PDF 深度阅读工具

痛点: 基于摘要的搜索无法获取实验细节(数据集、超参数、消融实验)，导致 idea 停留在综述层面。

解决方案:

- 集成 Unstructured.io 开源库解析 PDF 全文
- 对 Top-K 论文的核心 idea，自动下载并提取:
  - Methods 章节的实验设计
  - Results 章节的关键数据
  - Ablation Study 的变量分析
- 存储到向量数据库(FAISS)，供 Agent 深度检索

5.2 工具实现规范

每个工具封装为标准接口，示例 - Tavily 网络搜索:

```
class TavilySearchTool(Tool):
name = "web_search"
description = "搜索最新的网络信息和行业趋势(近 1 年内)"
```

```
def execute(self, query: str) -> dict:
    # 调用 Tavily API
    response = tavily_client.search(
        query=query,
        search_depth="advanced",
        max_results=5
    )
    # 返回清洗后的文本内容
    return {
        "results": [
            {"title": r.title, "content": r.content, "url": r.url}
            for r in response.results
        ]
    }
```

V2.0 新增 - PDF Reader Tool:

class PDFReaderTool(Tool):
name = "pdf_reader"
description = "下载并全文解析学术论文 PDF，提取实验细节"

```
def execute(self, arxiv_id: str, sections: list[str] = ["methods", "results"]) -> dic
    # 1. 从 arXiv 下载 PDF
    pdf_path = download_arxiv_pdf(arxiv_id)

    # 2. 使用 Unstructured 解析
    elements = partition_pdf(pdf_path)

    # 3. 提取目标章节
    extracted = extract_sections(elements, sections)

    # 4. 存入向量数据库
    store_to_faiss(arxiv_id, extracted)

    return {
        "arxiv_id": arxiv_id,
        "sections": extracted,
```

```
        "token_count": count_tokens(extracted)
    }
```

5.3 API 调用时机

- **Separate 阶段开始前**: 发散者和专家自动调用 `paper_search` 和 `web_search` 获取背景
- **Together 阶段批评环节**: 批评者调用 `citation_graph` 验证已有研究
- **Top-K 深化阶段**: 领域专家调用 `pdf_reader` 深度阅读核心论文(每篇≤10 分钟)
- **用户主动触发**: 通过指令"搜索相关论文"显式调用

5.4 速率限制与缓存

- **arXiv API**: 无速率限制，但建议每秒不超过 3 次请求
- **Semantic Scholar**: 每 5 分钟最多 100 次请求(免费层)
- **PDF 下载**: 使用 arxiv.org 的镜像源，单次最多下载 5 篇论文
- **实现本地缓存**: 相同查询 24 小时内直接返回缓存结果，减少重复调用

---

6. 技术实现细节

6.1 编排调度器(Orchestrator)

负责控制整个 Separate-Together 流程的状态机:

class Orchestrator:
def **init**(self, agents: list[Agent], blackboard: Blackboard):
self.agents = agents
self.blackboard = blackboard
self.phase = "separate" # separate | together | evaluate

```
def run(self, user_input: str, max_rounds: int = 3):
    for round in range(max_rounds):
        # 阶段一: Separate
        initial_ideas = self.separate_phase(user_input)

        # ⚠ V2.0 新增: 语义聚类分组
        idea_groups = self.cluster_ideas(initial_ideas, num_groups=4)
```

```
        # 阶段二: Together - 分组讨论
        for group in idea_groups:
            self.together_phase_grouped(group, iterations=3)

        # 全局整合
        synthesized_ideas = self.global_synthesis(idea_groups)

        # 阶段三: Tournament Evaluation
        top_k_ideas = self.tournament_evaluate(synthesized_ideas, k=5)

        if self.convergence_check(top_k_ideas):
            break

        # 下一轮深化 Top-K
        user_input = self.refine_prompt(top_k_ideas)

    return self.final_output()

def cluster_ideas(self, ideas: list[Message], num_groups: int) -> list[list[Messa
    """使用语义相似度将 ideas 聚类分组"""
    embeddings = [get_embedding(idea.content) for idea in ideas]
    labels = kmeans_clustering(embeddings, n_clusters=num_groups)

    groups = [[] for _ in range(num_groups)]
    for idea, label in zip(ideas, labels):
        groups[label].append(idea)
    return groups

def tournament_evaluate(self, ideas: list[Message], k: int) -> list[Message]:
    """锦标赛淘汰制评估"""
    # 初始化 Elo 分数
    elo_scores = {idea.id: 1500 for idea in ideas}

    # 每个 idea 进行 5-8 场对比
    for idea in ideas:
        for _ in range(6):
            opponent = random.choice([i for i in ideas if i.id != idea.id])
```

```
        # 评估者投票
        winner = self.evaluator_agent.compare(idea, opponent)

        # 更新 Elo
        if winner == idea.id:
            elo_scores[idea.id] += 16
            elo_scores[opponent.id] -= 16
        else:
            elo_scores[idea.id] -= 16
            elo_scores[opponent.id] += 16

    # 返回 Top-K
    ranked_ideas = sorted(ideas, key=lambda x: elo_scores[x.id], reverse=True)
    return ranked_ideas[:k]
```

6.2 上下文窗口管理

随着讨论轮次增加，黑板消息量膨胀可能超过 LLM 上下文限制(GPT-4o 为 128K tokens):

策略一 - 滑动窗口: 每个 Agent 只读取最近 N 条消息(N = 20)

策略二 - 摘要压缩: 每轮结束后，用 LLM 生成该轮摘要，替换原始消息

策略三 - 引用追踪: 通过 `references` 字段保留关键消息链，删除孤立消息

⚠ **V2.0 优化 - 分组隔离**: 分组讨论天然减少每个 Agent 的上下文负载(从 16 个 idea 降至 3-5 个)

6.3 并发与异步调用

- **Separate 阶段**多个 Agent 并行运行，使用 Python `asyncio` 实现
- **API 调用**采用异步非阻塞方式(aiohttp)，提升速度 3-5 倍
- **使用信号量控制并发数**，避免 API 速率限制触发

6.4 错误处理与容错

- **LLM API 失败**: 自动重试 3 次，失败后降级到备用模型(如 Claude)
- **外部搜索 API 超时**: 设置 10 秒超时，超时后跳过该工具调用
- **Agent 输出格式错误**: 使用 Pydantic 校验，不符合格式则要求 Agent 重新生成
- **循环失控**: 设置最大消息数阈值(100 条)，超过后强制进入评估阶段

---

7. 数据结构与存储

7.1 运行时数据结构

数据类型	存储方式
Agent 状态	内存(Python 对象)
黑板消息	内存 + 可选持久化(SQLite)
API 调用缓存	Redis(24 小时 TTL)
PDF 全文向量	FAISS 向量数据库
运行历史记录	JSON 文件 / PostgreSQL

7.2 可观测性与日志

- **结构化日志**: 每次 Agent 调用记录 `agent_name`, `input_tokens`, `output_tokens`, `latency`
- **消息追踪**: 每条黑板消息包含完整引用链，便于回溯推理路径
- **成本监控**: 实时统计 API 调用花费，达到预算阈值时发出警告

---

8. 用户交互设计

8.1 输入接口

用户通过三种方式启动系统:

1. **命令行接口(CLI)**: `python main.py --topic "多智能体在药物发现中的应用" --rounds 3`

2. **Web 界面**: 基于 Gradio 的简易 UI，输入框 + 实时输出流
3. **API 服务**: RESTful 接口，支持异步任务提交和结果查询

8.2 输出格式

系统输出分为三个层次:

- **实时流式输出**: 每个 Agent 发言时实时显示(WebSocket)
- **阶段性总结**: 每轮结束后输出该轮 Top-5 idea 和 Elo 分数
- **最终报告**: Markdown 格式完整报告，包含:
  - 原始研究主题
  - 最终精选的 3-5 个 idea(带详细描述和可行性分析)
  - 每个 idea 的引用论文和外部资料链接(含 PDF 深度阅读摘要)
  - Agent 讨论历史摘要
  - 成本统计(API 调用次数和费用)

8.3 人工干预机制

- 用户可在 Together 阶段暂停，手动注入新观点或约束
- 可手动调整某个 Agent 的 Persona 或温度参数
- 可强制终止当前轮次，直接进入评估

---

9. 非功能需求

9.1 性能指标

指标	目标值
单轮 Separate-Together 完成时间	≤5 分钟
并发 Agent 数量支持	≥10 个
系统内存占用	≤2GB(不含 LLM 调用)
API 调用成功率	≥95%(含重试)
PDF 解析速度	≤10 秒/篇

9.2 可扩展性

- **模型无关性**: 支持切换不同 LLM 提供商(OpenAI, Anthropic, 本地模型)
- **工具可插拔**: 新增外部 API 只需实现 Tool 接口
- **协作模式可配置**: 未来可扩展为 Evolutionary、Swarm 等其他模式

9.3 成本控制

- **单次运行预算上限**: 可配置(默认 $5)
- **优先使用免费 API**: arXiv, Semantic Scholar, 本地 PDF 解析
- **对高成本模型调用**(GPT-4o)设置请求缓存

9.4 安全与合规

- **API Key** 通过环境变量或加密配置文件管理，不硬编码
- **外部搜索结果**需过滤敏感信息(个人隐私、机密数据)
- **生成的 idea** 添加免责声明: 需人工验证可行性和伦理合规性

---

10. 开发计划与里程碑

10.1 MVP 范围(第一阶段，2 周)

- 实现 Agent 基础抽象和黑板通信
- 实现 Orchestrator 的 Separate-Together 流程
- 配置 4 个 Persona Agent(发散者、专家、批评者、评估者)
- 集成 OpenAI API 作为 LLM 引擎
- CLI 命令行交互
- 输出 Markdown 格式报告

10.2 增强版(第二阶段，3 周)

- 集成 Tavily, arXiv, Semantic Scholar API
- ⚠ **V2.0 新增**: 实现 PDF Reader Tool(基于 Unstructured)
- 实现 Cognitive Planning prompt 前置步骤
- ⚠ **V2.0 新增**: 实现锦标赛评估机制(Tournament Selection)
- ⚠ **V2.0 新增**: 实现语义聚类分组讨论
- 实现上下文窗口管理(摘要压缩)

- 添加并发调用和错误重试机制
- 实现 Gradio Web 界面

10.3 生产级(第三阶段，4 周)

- 支持多种 LLM 后端(Claude, DeepSeek 本地模型)
- 实现消息持久化和运行历史查询
- 添加成本监控和预算控制
- API 服务化(FastAPI + 异步任务队列)
- 完整的单元测试和集成测试
- 性能优化和压力测试

---

11. 验收标准

11.1 功能验收

1. 输入"多模态大模型在医疗诊断中的应用"，系统能在 5 分钟内输出 ≥3 个可行 idea
2. 每个 idea 包含至少 2 篇 2024-2026 年的论文引用
3. ⚠ **V2.0 新增**: Top-3 idea 包含至少 1 篇论文的深度阅读摘要(Methods/Results 章节)
4. 多次运行相同主题，产出的 idea 有 ≥50% 差异性(通过语义相似度检测)
5. Together 阶段能观察到明确的 Extension/Combination/Refinement 行为

11.2 技术验收

1. API 调用成功率 ≥95%(100 次测试)
2. 单次运行成本 ≤$2.00(基于 2026 年 3 月定价)
3. 并发 5 个用户请求时，系统稳定运行无崩溃
4. 代码覆盖率 ≥70%(核心模块)
5. ⚠ **V2.0 新增**: 锦标赛评估结果与传统打分的 Spearman 相关性 ≥0.7

11.3 质量验收

1. 输出 idea 经人工评审，新颖性评分 ≥7/10(5 位专家平均分)
2. 至少 1 个 idea 能够直接转化为可执行的研究方案
3. 用户满意度 ≥4/5(内部测试用户问卷)

---

12. 风险与挑战

12.1 技术风险

风险	影响	缓解措施
LLM API 不稳定	系统可用性下降	多模型备份，本地模型降级
上下文窗口溢出	信息丢失	智能摘要 + 分组讨论隔离
API 成本超预算	运营不可持续	缓存、预算上限、免费 API 优先
Agent 陷入循环辩论	无法收敛	最大轮次限制、强制收敛机制
PDF 解析失败	深度阅读功能失效	降级到摘要模式 + 人工审核

12.2 产品风险

- **idea 质量不稳定**: 依赖 Persona 设计和 prompt 质量，需持续优化
- **领域适应性有限**: 某些高度专业化领域(如理论物理)可能缺乏足够外部知识
- **用户学习成本**: 需提供详细文档和示例，降低使用门槛

---

13. 未来扩展方向

- **多模态支持**: 集成 GPT-4V 等模型，支持图表、实验数据的理解
- **知识图谱增强**: 构建领域知识图谱，提供更精准的概念关联
- **协作模式库**: 预设 Evolutionary、Swarm 等多种协作模式供用户选择
- **社区 Persona 市场**: 允许用户分享和下载定制化 Persona 配置

- **实验管理集成**: 将生成的 idea 直接导出为实验计划和文献综述草稿
- ⚠ **V2.0 规划**: 实时协作模式 - 多位人类研究者与 AI Agent 共同参与讨论

附录 A: V2.0 核心改进总结

A.1 架构层面

1. **黑板消息分拣机制**: 将 10-16 个初始 idea 按语义聚类分为 3-4 组，每组独立讨论后再全局整合
2. **分组讨论隔离**: Together 阶段分组并行执行，减少单个 Agent 的上下文负载(从 16 个降至 3-5 个)

A.2 评估机制

3. **锦标赛淘汰制**: 替代绝对打分，使用两两对比 + Elo 积分系统，提升 LLM 评估稳健性

A.3 知识深度

4. **PDF 全文解析工具**: 基于 Unstructured.io 实现 Top-K 论文的深度阅读，提取实验设计和关键数据
5. **向量数据库存储**: 使用 FAISS 存储论文全文，支持 Agent 精准检索实验细节

A.4 技术优化

6. **语义聚类算法**: 使用 K-Means + Embedding 实现 idea 自动分组
7. **分布式并行**: 分组讨论天然支持并行化，理论上可将 Together 阶段时间缩短 50%

参考文献

[1] Tavily API. (2024). Real-Time Web Knowledge for Custom LLM Apps. https://tavily.com

[2] MultiAgentBench. (2025). Evaluating the Collaboration and Competition of LLM agents. arXiv:2503.01935

[3] Persona-based Multi-Agent Collaboration for Brainstorming. (2025). arXiv:2512.04488

[4] 多智能体系统设计: 协作、竞争与涌现行为. (2025). 腾讯云开发者社区

[5] Creative as IDEO experts: LLM-agent-based design thinking workshop. (2025). ICCC 2025

[6] Multi-LLM-Agents Debate - Performance, Efficiency, and Scaling Challenges. (2025). ICLR Blog Posts

[7] Multi-Agent Systems Powered by Large Language Models: Applications in Swarm Intelligence. (2024). arXiv:2503.03800

[8] Unstructured.io. (2024). Open-Source PDF and Document Parsing Library. https://unstructured.io

[9] Elo Rating System. (1978). The Rating of Chessplayers, Past and Present. Arco Publishing.